

EncryptorX Steganography Engine

Cryptographic Steganography with Hull-Dobell Pixel Shuffling & Preamble Bootstrapping

Document Version: 2.0.0 | Classification: Technical Whitepaper & Specification | Reference Implementation: Python encryptorx.stego / Rust GUI | Date: September 2026

Executive Summary: Conventional Least Significant Bit (LSB) steganography embeds data sequentially from pixel (0,0), creating localized spatial artifacts and statistical step-signatures readily detectable via Chi-Square and Sample-Pair steganalysis. EncryptorX Stego introduces **Pseudo-Random Pixel Shuffling** governed by the *Hull-Dobell Theorem*, achieving full-period cycle-walking permutations with strict $O(1)$ memory overhead. Payloads are shielded with **PBKDF2-HMAC-SHA256** key derivation, stream keystream encryption, and **Encrypt-then-MAC (HMAC-SHA256)** integrity verification. A universal 48-bit preamble bootstrapping format enables automated carrier inspection, multi-bit allocation (1, 2, or 4 bpc), and selective channel targeting (RGB, Blue-only).

1. Steganalysis Vulnerabilities & Threat Model

Digital steganography aims to conceal the very existence of a communication channel within an innocent cover medium. Traditional sequential LSB replacement fails against modern automated detection pipelines due to three critical flaws:

- Spatial Density Clustering:** Sequential injection concentrates bit flips in the top-left corner of an image. Visual differential maps reveal an abrupt rectangular boundary where the payload ends.
- Pair of Values (PoV) Distortion:** Natural image pixels feature smooth histograms. LSB modification couples pixel pairs $(2k, 2k+1)$, tending their counts toward equal frequencies. Westfeld and Pfitzmann's Chi-Square analysis detects this deviation with $p < 0.001$.
- Absence of Authenticity:** Standard steganography tools lack cryptographic MACs. An adversary can perform bit-flipping or payload truncation without detection by the receiver.

2. Mathematical Foundations: Hull-Dobell Permutations

To scatter bits uniformly across the carrier without maintaining an in-memory lookup table of millions of pixel coordinates, EncryptorX utilizes a pseudo-random Linear Congruential Generator (LCG) configured to guarantee a full period over power-of-two domains.

Theorem (Hull & Dobell, 1962): A linear congruential sequence defined by $X_{n+1} = (a \cdot X_n + c) \bmod M$ possesses a full period of length M if and only if:

- $\text{gcd}(c, M) = 1$ (increment and modulus are coprime).
- Every prime factor of M divides $a - 1$.
- If M is a multiple of 4, then $a \equiv 1 \pmod{4}$.

Cycle-Walking Permutations:

Let N be the total available channel slots ($W \times H \times \text{Channels}$). We compute $k = \lceil \log_2(N) \rceil$ and set modulus $M = 2^k \geq N$. We derive multiplier $a = (\text{seed} \cdot 8 + 5) \bmod M$ and increment $c = (\text{seed} \cdot 2 + 1) \bmod M$. By construction, c is odd and $a \equiv 1 \pmod{4}$. When $X_n \geq N$, the generator discards the sample and advances (Cycle-Walking). Because $M < 2N$, the expected number of steps per valid slot is strictly < 2 , maintaining $O(1)$ memory overhead.

3. Visual Fidelity Metrics & Human Visual System (HVS)

The objective visual transparency of stego images is evaluated using Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR):

Mean Squared Error: $MSE = (1 / 3WH) \cdot \sum_{x,y,c} [I(x,y,c) - I'(x,y,c)]^2$

Peak Signal-to-Noise Ratio: $PSNR = 10 \cdot \log_{10}(255^2 / MSE) \quad [dB]$

In 1-bit LSB mode, each targeted channel value is modified by at most +/- 1. For typical payloads occupying < 20% of image slots, **PSNR exceeds 65 to 75 dB**, which is mathematically indistinguishable from camera CMOS sensor thermal noise.

Blue-Channel Invariance: The human retina contains S-cones (blue sensitive) accounting for only ~5-7% of total foveal photoreceptors. Targeting only the Blue channel (channels='B') ensures complete human visual imperceptibility even at 2 or 4 bits per channel.

4. Binary Frame Protocol & Preamble Bootstrapping

EncryptorX partitions embedding into two deterministic phases to allow automated, zero-knowledge extraction:

Segment	Size	Embedding Rules	Purpose / Encoding Description
MAGIC HEADER	4 bytes	Fixed 1 bpc, RGB	Identifier b'SX02' (0x53 0x58 0x30 0x32)
FLAGS	1 byte	Fixed 1 bpc, RGB	Bit 0: Is_File Bit 1: Is_Password_Protected Bit 2: Compressed
CONFIG	1 byte	Fixed 1 bpc, RGB	Bit 7: Algorithm (0=LCG, 1=PRNG) Bits 4-6: Channels Bits 0-3: BPC
SALT	16 bytes	Configured bpc & chan	CSPRNG cryptographic salt for PBKDF2 key derivation
METADATA	Var (1+N)	Configured bpc & chan	Original filename length (1 byte) + UTF-8 filename (if file flag set)
PAYLOAD LEN	4 bytes	Configured bpc & chan	Big-endian unsigned 32-bit integer length of ciphertext
NONCE / IV	16 bytes	Configured bpc & chan	CSPRNG initialization vector for stream cipher keystream
CIPHERTEXT	N bytes	Configured bpc & chan	XOR stream keystream encrypted payload (raw or zlib-compressed)
HMAC TAG	16 bytes	Configured bpc & chan	HMAC-SHA256 (Encrypt-then-MAC) truncated 128-bit integrity tag

5. Cryptographic Security & Separation of Keys

EncryptorX adheres to the rigorous **Encrypt-then-MAC** paradigm, guaranteeing defense against Chosen-Ciphertext Attacks (CCA2):

- Key Derivation Function (KDF):** Password and salt are processed through PBKDF2-HMAC-SHA256 with 100,000 iterations to produce a 64-byte master key material. Two independent 32-byte keys are derived: K_{enc} for confidentiality and K_{auth} for authentication.
- Permutation Seed Isolation:** The shuffling seed is derived via SHA-256 with domain separation (seed = SHA256(password + b':PREAMBLE_INIT')). An attacker without the password cannot reconstruct the spatial permutation.
- Anti-Tampering Enforcement:** Prior to decryption, the HMAC tag is verified in constant time via hmac.compare_digest. If any image pixel harboring stego bits has been altered, extraction halts immediately with AuthenticationError.

6. Configurable Modes & Empirical Capacity Matrix

Image Resolution	Mode	Total Pixels	Channels	Net Capacity	Avg PSNR	Stealth Classification
512 × 512 (Square)	1 bpc	262,144	RGB (3)	98,240 B (~96 KB)	> 65 dB	Ultra Stealth (Undetectable)
512 × 512 (Square)	2 bpc	262,144	RGB (3)	196,544 B (~192 KB)	> 52 dB	High Stealth (Balanced)
512 × 512 (Square)	4 bpc	262,144	RGB (3)	393,152 B (~384 KB)	> 38 dB	Maximum Density
1920 × 1080 (Full HD)	1 bpc	2,073,600	RGB (3)	777,536 B (~759 KB)	> 68 dB	Ultra Stealth (Undetectable)
1920 × 1080 (Full HD)	1 bpc	2,073,600	Blue-Only (1)	259,136 B (~253 KB)	> 72 dB	Maximum HVS Invisibility

3840 × 2160 (4K UHD)	2 bpc	8,294,400	RGB (3)	6,220,736 B (~5.93 MB)	> 54 dB	High Density Archive
----------------------	-------	-----------	---------	------------------------	---------	----------------------

7. Developer API Reference & CLI Usage

```
# Python 3.8+ API Example:
from encryptorx.stego import hide_text, extract_text, calculate_metrics, StegoEngine

# 1. High-level conceal and extract
out_img = hide_text('carrier.png', 'Secret msg', 'stego.png', password='Key', bits_per_channel=1)
recovered = extract_text('stego.png', password='Key')
metrics = calculate_metrics('carrier.png', 'stego.png')
print('PSNR: ' + str(metrics['psnr_db']) + ' dB | MSE: ' + str(metrics['mse']))

# 2. CLI commands:
python -m encryptorx.stego hide-text -i photo.png -t 'Message' -o out.png -p 'Key123'
python -m encryptorx.stego extract-text -i out.png -p 'Key123'
python -m encryptorx.stego hide-file -i photo.png -f document.pdf -o out.png -b 2
python -m encryptorx.stego extract-file -i out.png -o ./recovered
python -m encryptorx.stego metrics -orig photo.png -stego out.png
```

Rust GUI Integration: Native desktop application compiled with eframe / egui. Tab 4 provides full point-and-click steganographic capabilities with zero console popups and live metric displays.